Knowledge Graph Completion with R-GCN and CompGCN: Link Prediction on Large-Scale Knowledge Graphs

Big Data Course Presentation

Charan Ponaganti (240150024)
M Charan (240150018)
Lalit Deshmane (240150044)

Mehta Family School of Data Science and Artificial Intelligence
Indian Institute of Technology, Guwahati

November 2026



Outline

1. Problem & Data
2. Models & Training
3. Results & Lessons

The Problem: Completing a Knowledge Graph

Knowledge graph completion (link prediction)

- Facts stored as triples (h, r, t) — head, relation, tail.
- Real KGs are incomplete; many true links are missing.
- Task: given $(h, r, ?)$ or $(?, r, t)$, rank entities so the true answer is near the top.
- Metrics: MRR and Hits@$\{1, 3, 10\}$.

Our goal

Compare relational GNN encoders (R-GCN, CompGCN) against shallow embeddings on a real, large biomedical KG — with Spark for ETL.

Why This Is a Big Data Problem

Dataset	$\lvert E\rvert$	$\lvert R\rvert$	Train
ogbl-biokg	94k	51	4.76M
ogbl-wikikg2	2.5M	535	16M
Wikidata5M	4.6M	822	21M
FB15k-237	15k	237	272k

Constraints

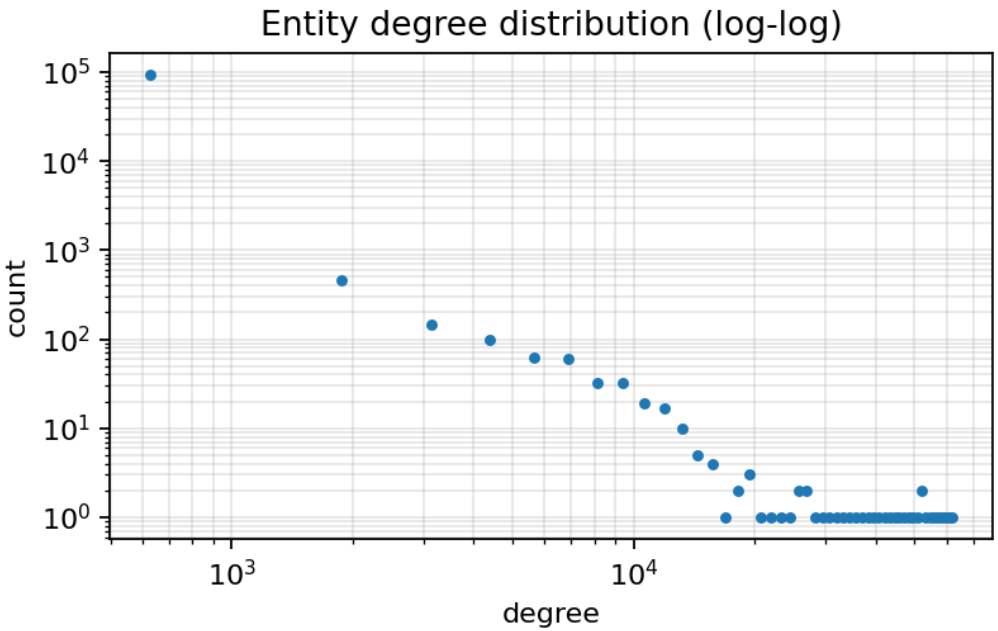
- Spark for ETL (course req.)
- Kaggle T4: 16 GB VRAM
- Full-batch GNN on millions of entities does not fit

Decision: train on `ogbl-biokg` with neighbor-sampled mini-batches; keep Wikidata5M as a Spark-only option.

Dataset: ogbl-biokg

After Spark cleaning

- 93,773 entities
- 51 relations (all N–N)
- 4,762,676 train triples
- median deg. 22, max 62,733
- 5 biomedical entity types



Heavy-tailed degrees ⇒ hubs make full 2-hop expansion unsafe.

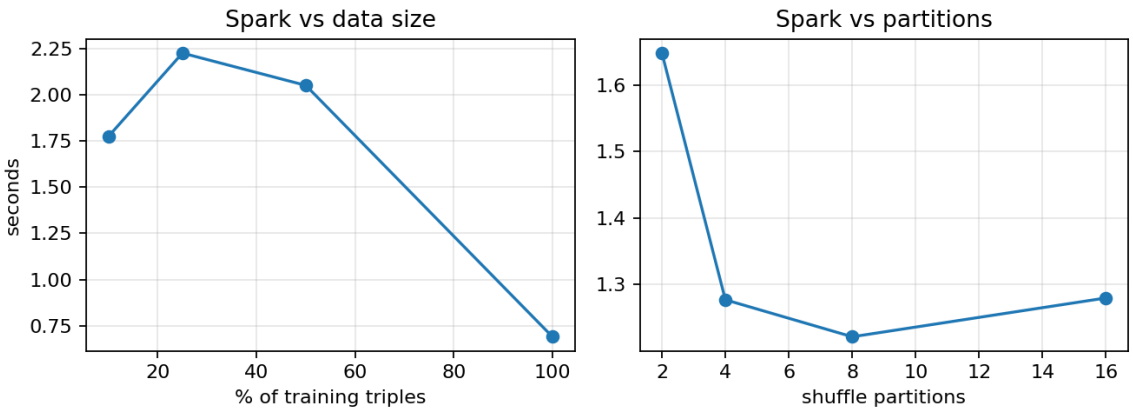
PySpark Pipeline

Stages (then `spark.stop()`)

1. Ingest with explicit (h, r, t) schema
2. Clean nulls, duplicates, self-loops
3. Profile degrees & Bordes relation categories
4. Dense IDs + inverse edges (102 message relations)
5. Build int64 keys for filtered ranking
6. Hand tensors to PyTorch; stop the JVM

Config: `local[*]`, Arrow, Kryo, 12 GB driver, 8 partitions.

Spark Scalability



Takeaways

- ETL profiling in $\approx$1–2 s
- 100% fastest = cache hit
- Partitions 2$\rightarrow$16 are flat

Models We Compare

DistMult

$f = \sum_k h_k r_k t_k$
Shared decoder + shallow baseline

R-GCN

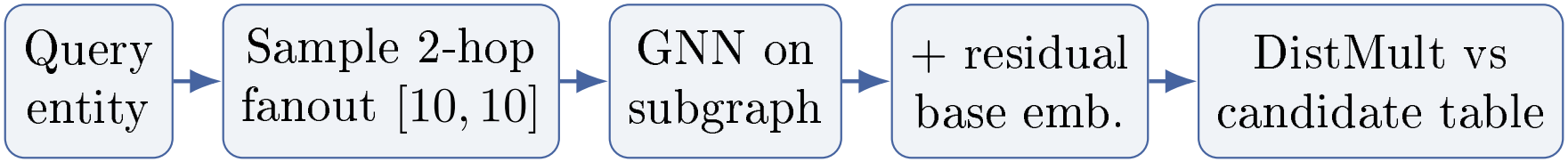
Per-relation W_r
Basis decomp. (B=8)
2 layers, dim 128

CompGCN

$\phi(h, r)$: sub / mult / corr
Direction-specific W
Relation update each layer

Extra shallow baselines (same budget): TransE and ComplEx.

Making GNNs Fit on a 16 GB T4



- Query-side encoding only; candidates from `model.ent`
- Mixed precision; CompGCN composition forced to float32
- OOM fallback: halve batch & fanout

Training Recipe

Design choices

- Bidirectional: $(h, r, ?)$ and $(?, r, t)$
- **Matched budget** for all 7 models
- Type-aware negatives (same entity type)
- Self-adversarial weighting (RotatE-style)
- Early stop on val. MRR (patience 5)

T4 preset

Epochs × steps	12 × 6
Negatives / batch	16 / 5
Dim / layers / bases	128 / 2 /
Fanout	[10,
Wall clock	1.7

Two Evaluation Protocols

Filtered candidates

- True vs 500 random entities
- Other truths masked (Spark keys)
- Head + tail, 15,000 triples
- Used each validation epoch

Official OGB

- Type-constrained negatives
- 162,870 test triples
- Comparable to literature
- **Main reported metric**

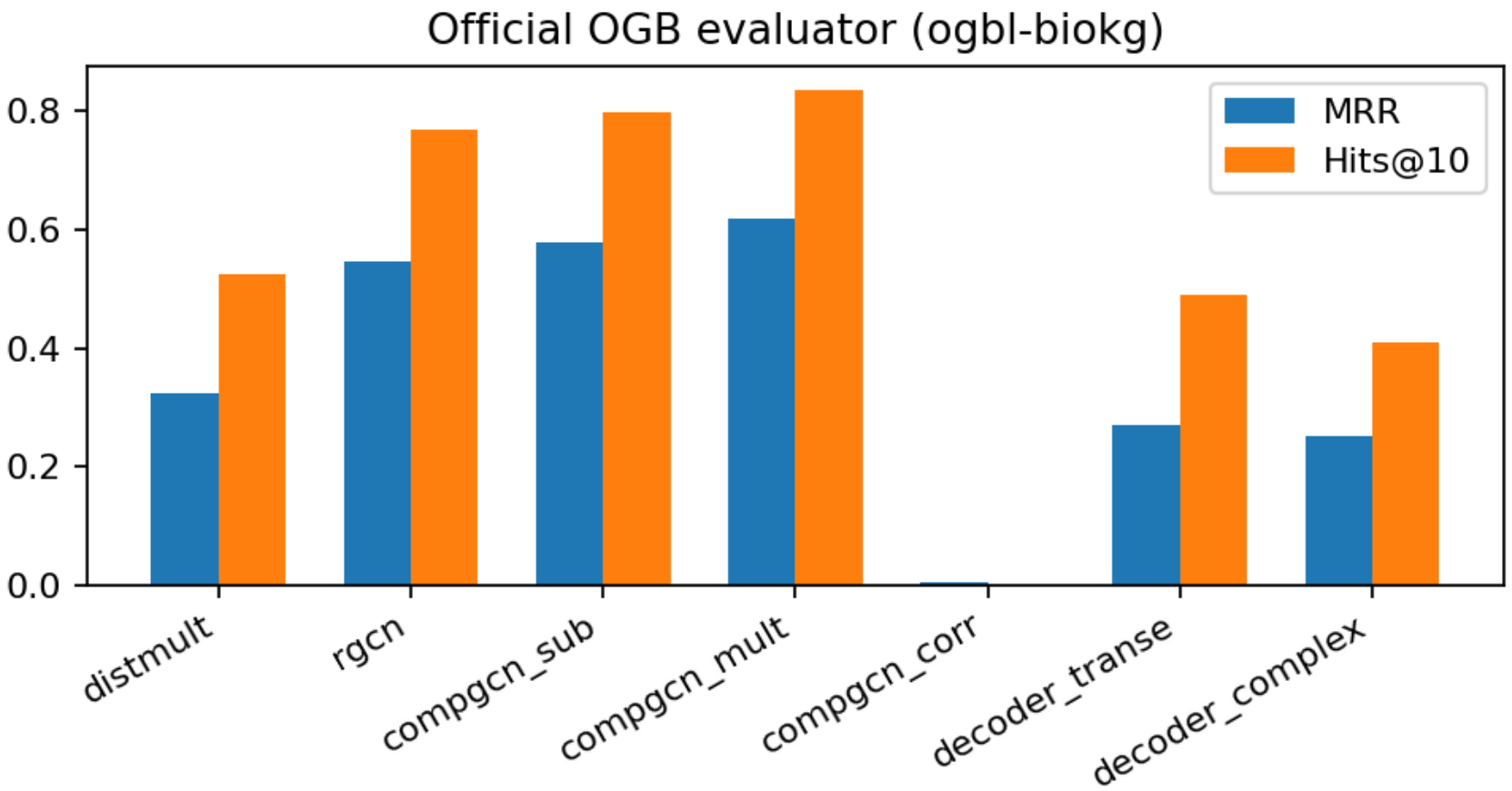
Lesson: OGB negatives are row-aligned; Spark reorders rows — evaluate OGB on the original test order.

Main Result: Official OGB Evaluator

Model	MRR	H@1	H@3	H@10
DistMult	0.325	0.224	0.364	0.526
R-GCN	0.547	0.434	0.606	0.768
CompGCN-sub	0.578	0.464	0.641	0.799
CompGCN-mult	**0.620**	**0.505**	**0.691**	**0.835**
CompGCN-corr	0.004	0.000	0.000	0.000
TransE	0.270	0.165	0.305	0.490
ComplEx	0.250	0.168	0.278	0.410

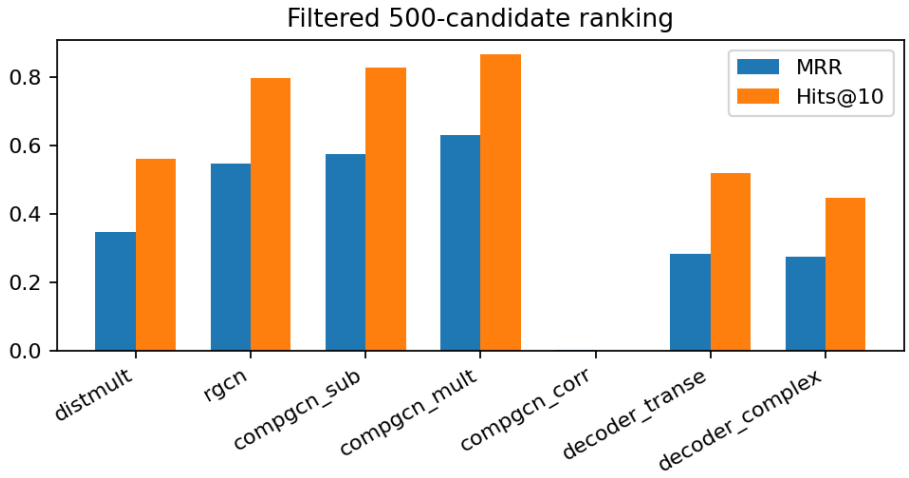
CompGCN-mult > CompGCN-sub > R-GCN > TransE/ComplEx > DistMult

OGB Metrics (Visual)



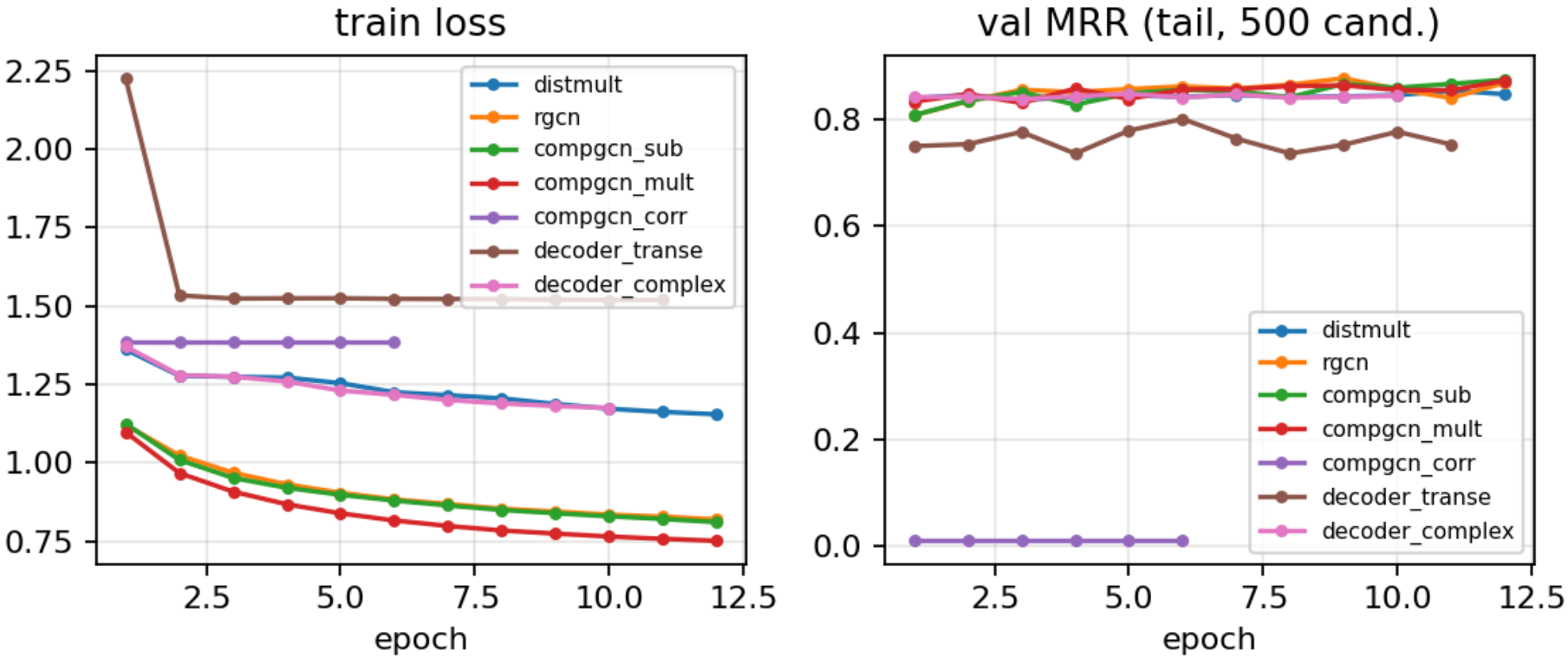
Second Protocol Agrees (500 Candidates)

Model	MRR	H@10	
DistMult	0.348	0.562	
R-GCN	0.549	0.798	
CompGCN-sub	0.576	0.828	
CompGCN-mult	**0.631**	**0.867**	
CompGCN-corr	0.002	0.000	
TransE	0.285	0.519	
ComplEx	0.275	0.448	110.6



Same ranking as OGB; matched budget removed the earlier DistMult artifact.

Training Dynamics



Lowest loss: CompGCN-mult. CompGCN-corr collapses after epoch 1 (early-stopped at epoch 6); all others run 12 epochs.

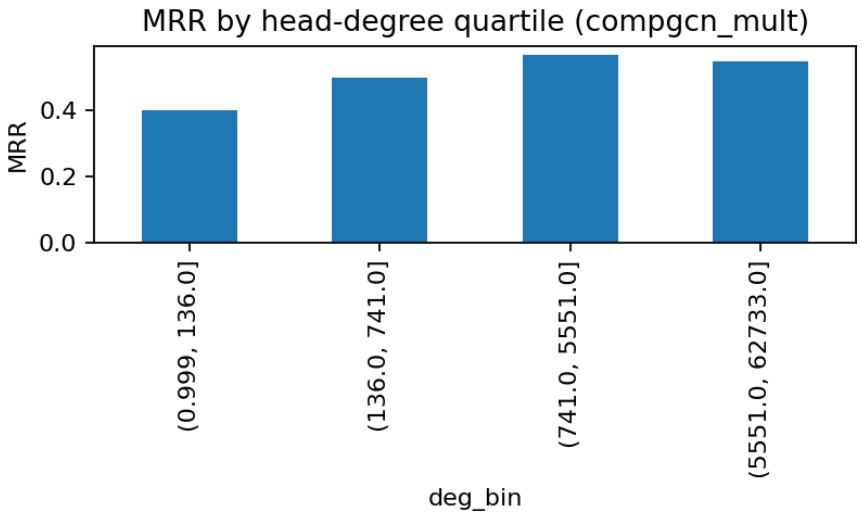
R-GCN Ablations (Half Budget)

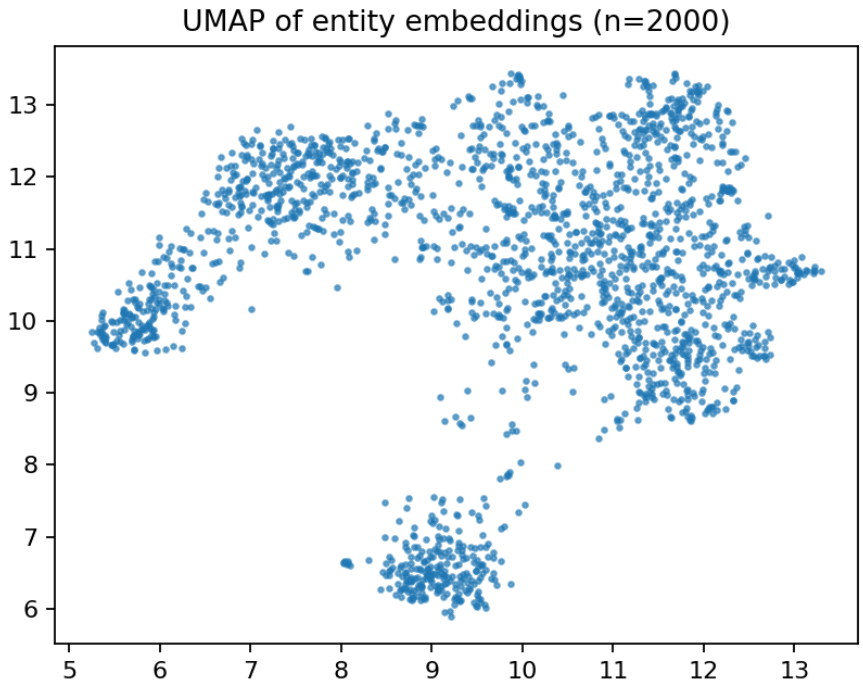
Setting	MRR	H@1	H@1	MR
Base (128, L2, B8, n16)	0.411	0.291	0.65	
1 layer	**0.449**	**0.323**	0.70	
8 negatives	0.332	0.213	0.58	
4 bases	0.395	0.270	0.64	
dim 64	0.436	0.316	0.67	

Takeaways

- 1 layer strongest
- 8 negatives hurts most
- dim 64 still competitive

Error Analysis: Rare Entities Are Hard





CompGCN-mult: MRR 0.401 (rarest quartile) vs 0.568 (best bin). All 51 relations are N–N $\Rightarrow$ relation-type breakdown uninformative.

Engineering Lessons

- **OGB misalignment:** Spark reordered test rows; evaluate on original OGB order.
- **Half-precision FFT:** CompGCN-corr broke under AMP; composition now float32.
- **Kernel deaths:** per-row negatives → millions of tensors; vectorize by entity type.
- **CUDA 13 installs:** compiling `torch-scatter` kills the kernel; binary wheels only.

Conclusion

Results

- Spark ETL in seconds
- Sampled GNNs fit a T4
- CompGCN-mult best (OGB MRR 0.620)
- GNNs $\gg$ shallow at equal budget

Limits

- Below published long-schedule SOTA
- Candidates not GNN-encoded
- CompGCN-corr unstable
- Spark on one node

Future

- Encode both sides
- ConvE decoder
- Stabilize corr
- Multi-node Spark / Wikidata5M

Thank You — Questions?

Knowledge Graph Completion with R-GCN / CompGCN

Code, notebook, results and report:

`https://github.com/Charanponaganti/Knowledge-Graph-Completion-with-R-GCN-CompGCN`

Charan Ponaganti M Charan Lalit Deshmane